Krovni izrek: $T(n) = aT(\frac{n}{b}) + O(n^d)$

$$T(n) = \begin{cases} O(n^d) & ; a < b^d \\ O(n^d \ln(n)) & ; a = b^d \\ O(n^{\log_b(a)}) & ; a > b^d \end{cases}$$

Iskanje z bisekcijo $O(\log(n))$

Vhod: urejena tabela a, x

izhod: ali je x v tabeli

```
def bisekcija(a,x)
    l,r := 0, len(a)-1
    while l<r :
        m = (l+r)//2
        if a[m]=x: return m
        if a[m]<x : l=m+1
        else: r=m-1
    return False
```

Urejanje z zlivanjem $O(n \log(n))$

```
def merge (a,b):
    if len(a)==0:
        return b
    if len(b)==0
        return a
    if a[1] ≤ b[1]:
        return a[1] + merge(a[1:], b)
    else: return b[1]+merge(a,b[1:])
```

```
def merge_sort(a):
    if n>1
        return merge(ms(a[:n/2], a[..]))
    else: return a
```

Izbira pivota

```
def select(k, a)   O(n)
    if len(a) == 1:
        return S[1]

    izberi i ∈ [n]
    p = S[i]
    S_L = a[:l]
    S_V = a[l+1:m]
    S_R = a[m+1:]

    if l+1 ≤ k ≤ m:
        return i

    if k ≤ l:
        return select(k, S_L)
    if k ≥ m+1
        return select(k-m, S_R)
```

```
def drug_select(a):
    izberi i ∈ [n]
    S_L, S_V, S_R

    if |S_L|, |S_R| ≤ 3/4 n
        return i
```

Hitro urejanje

```
def quick sort (a)
    if len(a) ≤ 1: return a
    p = random.choice(a)
    S_L, S_V  S_R
    return quick_sort(S_L)
           + S_V
           + quick_sort(S_R)
```

DFS

lahko usmerjen $O(|V|^2)$ $(V^2 \geq E)$

```
def dfs(G, v)   O(E)
    visited[v] := True
    s := sklad
    s.push(v)
    while s:
        u := s.pop()
        if not visited[u]
            for w in sosedi(u)
                s.push(w)
            visited[u] = True
```

- povezana komponenta od v
- pre-post
- usmerjen cikel $\Leftrightarrow$ povratna povezava

$u \longrightarrow v$

prema	$[a,b] \supset [c,d]$
povratna	$[a,b] \subset [c,d]$
prečna	$[a,b] \cap [c,d] = \emptyset$

- Topološka urejenost (če je DAG) (uredimo po največjem post)

Krepko povezane komponente

$u \rightsquigarrow v$, $v \rightsquigarrow u$

- $C(G) = C(G^R)$
- kvacientigraf je DAG
- $C \to C' \Rightarrow \max post(C) > \max pos(C')$
- točka z največjim post pri DFS je v ponoru

Tarjanov algoritem (za krepko povezane komponente)

konstruiraj G^R $(O(V+E))$

Razvrstimo z DFS po padajočem post

Izvedemo DFS za G,

iz ponora naprej

BFS $O(V+E)$

```
def bfs(G,s)
    raz = [∞]*n
    raz[s] = 0
    Q = vrsta
    Q.push(s)
    while Q:
        u = Q.pop()
        for u∈sosed(v)
            if raz[v] = ∞:
                raz[v] = raz[u]+1
                Q.push(v)
```

Dijkstra $O((V+E)\log(V))$

```
def Dijkstra(G,l,s)
    dist=[∞]*n
    prev=[none]*n
    dist[s]=0
    Q = prioritetna vrsta
        u = Q.getmin()
        for v sosed(u):
            if dist(v) = dist(u)+ω(u,v):
                dist(v) = dist(u)+ω(u,v)
                prev(v)=u
            Q.decrease_key(v,dist(v))
```

prioritetna vrsta:

insert	$O(\log(V))$
extract_min	$O(\log V)$
decrease_key	$O(\log V)$

(Dvojiška kopica)

Topološka ureditev

```
def toposort(G)        O(V+E)
    indeg = [0]*n
    for i in range(n)
        for j in sosedi(i)
            indeg[j]+=1
    vrsta = tisti, ki imajo 0 staršev
    topo = []
    while vrsta:
        c=vrsta.popleft()
        topo.append c
        for j in sosedi(c)
            indeg[j] -=1
            if indeg[j]==0
                vrsta.push(j)
    return topo
```

```
def deg-najkrajša_pot (G,s)        O(V·deg(G))
    dist = [∞]*|V|
    prev = [None]*|V|
    dist[s]=0
    topološko_uredi(G)
    for u∈V (topološko urejeni)
        for u,w sosed(v)
            if dist(v)>dist(u)+w:
                dist(v)=dist(u)+w
                prev(v)=u
```

Bellman-Ford $\mathcal{O}(V \cdot E)$

vhod: usmerjen graf, brez negativnih ciklov, s

izhod: razdalja do s za vsako vozlišče

```
def Bellman_ford (G, S)
    dist = [∞] * |V|
    prev = [None] * |V|
    dist[s] = 0
    for _ in range(|V|-1)
        for (u,v,w) ∈ E
            if dist(v) > dist(u) + w:
                dist(v) = dist(u) + w
                prev(v) = u
```

Floyd-Warshalov $\mathcal{O}(V^3)$

$D_k(i,j)$ ... najkrajša pot od i do j, z vozlišči $[k]$

$$D_0(i,j) = \begin{cases} w(i,j) \\ \infty \end{cases}$$

$$D_k(i,j) = \{ D_{k-1}(i,k) + D_{k-1}(k,j),\ D_{k-1}(i,j) \}$$

$D_n(i,j)$ najkrajša razdalja

$D_n(i,i) < 0 \Rightarrow$ negativni cikel

Požrešna metoda

znaki

frekvence znakov

```
def Huffmanova_kopice(S,f)
    H...prioritetna vrsta urejena po f
    for k in rang(len(f), 2len(f))
        i,j = dva najmanjša iz H
        f[k] = f[i] + f[j]
        ustvarimo node z i,j otroboma
        H.insert(f[k],k)
```

Drevesa

$T(G) = T(G-e) + T(G/e)$

```
def kruskov_algoritem(G)
    T=∅
    for v ∈ V
        Make-Set(v)
    sort(E)
    for (w,u,v) ∈ E
        if Find-set(u) ≠ Find-Set(v)
            T.append((w,u,v))
            Union(u,v)
    return T
```

Unija z rangom

```
def Makeset(x)        O(1)
    π(x) = x
    rank = 0
def Find(x)           O(log n)
    while x ≠ π(x)
        x = π(x)
def union(x,y)        O(log n)
    rx = Find(x)
    ry = Find(y)
    if rx == ry
        return
    if rank(rx) > rank(ry)
        π(ry) = rx
    if rank(rx) == rank(ry)
        rank(ry) = rank(ry) + 1
    else: π(rx) = ry
```

```
def kompresija_poti_find(x)
    if x ≠ π(x):
        π(x) = komp_po_find(π(x))
    return π(x)
```

$O(m \log^*(n))$

$\log^*(n) = \underbrace{\log(\log(\ldots(\log(n))}_{?}) \leq 1$

```
def primov_algoritem(G)
    Q = V
    kljuc = [∞]*len Q
    kljuc(r) = 0
    π(r) = None
    while Q ≠ ∅:
        u = Q.Extract-min
        for v ∈ sosedi(u)
            if v∈Q ∧ w(v,u) < kljuc[v]
                π[v] = u
                kljuc[v] = w(u,v)
```

Q kopica

Binarna:

$O(E \cdot \log V)$

Fibon:

$O(V \cdot \log V + E)$